Automated Network Request Management in ServiceNow

A Project Report

Submitted by

Pathan Asif Khan

Department of Artificial Intelligence and Data Science

Seshadri Rao Gudlavalleru Engineering College

Project Domain: ServiceNow / IT Service Management / Network Request Automation

Platform: ServiceNow Personal Developer Instance (PDI)

Program: SmartBridge

Table of Contents

1. Introduction

1.1 Project Overview

Automating network request management in ServiceNow enhances operational efficiency by streamlining workflows, reducing manual interventions, and ensuring real-time updates. The project implements a network change/request management process in ServiceNow to handle user-initiated network requests, route them through approval workflows, create fulfillment records, maintain audit-ready records, and provide request tracking.

The solution is built on a ServiceNow Personal Developer Instance (PDI) and centres on a Service Catalog item that captures request details, a Flow Designer flow that moves submitted data into a custom backend table, and an approval and fulfillment path that routes work to the appropriate network team.

1.2 Purpose

The purpose of the project is to:

- Streamline network request submission.
- Reduce manual intervention.
- Standardize network request information.
- Automate approval routing.
- Automate fulfillment.
- Improve request visibility.
- Provide status tracking.
- Maintain auditability.
- Improve operational efficiency.
- Support reporting and monitoring.

Key stakeholders of the solution are:

- End Users / Requesters
- Approvers
- Network Fulfillment Team
- IT Administrators
- Developers / System Administrators

2. Ideation Phase

2.1 Problem Statement

Network access and connectivity requests raised through informal or manual channels are difficult to standardize, track, and audit. Without a structured intake and workflow, requests can lack consistent information (device details, business justification, requester identity), approval steps can be skipped or

delayed, and there is no single source of truth for request status or history. This makes it hard for administrators to monitor request volume, approval turnaround, or fulfillment performance.

2.2 Empathy Map Canvas

Purpose: Illustrates the perspective of the requester and network team stakeholders during the ideation phase.

Explanation: This canvas should be inserted as originally produced during the ideation phase of the project.

2.3 Brainstorming

Purpose: Shows the ideas generated during the brainstorming phase that fed into the solution requirements.

Explanation: Insert the brainstorming board/canvas produced during the ideation phase.

3. Requirement Analysis

3.1 Customer Journey Map

The end-to-end journey of a network request, as defined for this project, is:

- Requester
- Network Request submission
- Validation
- Approval
- Fulfillment
- Network Team processing
- Completion
- Notification
- Tracking / Reporting

Purpose: Maps the requester's experience across each stage of the network request lifecycle.

Explanation: Insert the Customer Journey Map canvas if produced during the ideation/requirement phase.

3.2 Solution Requirement

Functional Requirements

Requirement	Status
Configure a Network Request catalog item in the Service Catalog.	Implemented

Requirement	Status
Capture network request information (request type, access level, device information, business justification, requester information, supporting documentation).	Partially implemented — variables captured are listed in Section 4.5; full field list should be confirmed against final screenshots.
Configure administrator-level components (Network Team group, approver groups/users, roles, access controls).	Network Team group confirmed as an active group. Approver groups/roles/ACLs: Not explicitly verified in the provided implementation evidence.
Implement dynamic catalog behaviour using Catalog UI Policies and Catalog Client Scripts.	Catalog UI Policy implemented (see Section 4.5). Catalog Client Script usage: Not explicitly verified in the provided implementation evidence.
Allow users to submit network requests through the Service Portal / Service Catalog.	Implemented
Automatically process submitted requests through Flow Designer.	Implemented — Flow Designer used as the primary automation mechanism (Section 4.7).
Retrieve catalog variables from the submitted Requested Item.	Implemented — Get Catalog Variables action (Section 4.7).
Create the appropriate backend/custom record from the submitted request.	Implemented — Create Record action populates the Network Database table.
Route requests for approval based on configured approval logic.	Implemented at a basic level — Ask for Approval / If-Else branching (Section 4.7). Specific approval routing rules (manager vs. security vs. group approval): Planned / Intended Feature — not confirmed as implemented.
Send automated email notifications to relevant users.	Implemented — Send Email action in the flow. Exact notification templates/triggers: Not explicitly verified in the provided implementation evidence.
Create/update the fulfillment record and assign it to the appropriate network team.	Assignment Group reference configured on the backend table; Not explicitly verified in the provided implementation

Requirement	Status
	evidence. for a distinct fulfillment task record.
Allow network engineers/team members to process the fulfillment work.	Planned / Intended Feature — not confirmed as implemented.
Maintain request status (New, Awaiting Approval, In Progress, Completed, Rejected, Cancelled).	Status is tracked via the Work Status field on the backend table; the specific set of status values implemented: Not explicitly verified in the provided implementation evidence.
Notify requesters at important stages.	Partially implemented via Send Email; full multi-stage notification coverage: Not explicitly verified in the provided implementation evidence.
Maintain auditability and request history.	Supported structurally by the Approval Request relationship/related list on the backend table; full audit reporting: Not explicitly verified in the provided implementation evidence.
Provide tracking of requests, approvals, fulfillment performance, and SLA information.	Planned / Intended Feature — not confirmed as implemented.

Non-Functional Requirements

- Maintainability — minimal reliance on custom Script Includes or Business Rules, favouring Flow Designer.
- Reusability — catalog variables and variable sets designed for reuse across catalog items.
- Usability — conditional UI behaviour (Catalog UI Policy) keeps the request form focused and relevant.
- Auditability — related-list linkage between requests and approvals for traceability.
- Ease of handoff — configuration intended to be understandable by future administrators/developers.

3.3 Data Flow Diagram

The flow of information through the system is:

User → Catalog Item → Requested Item (RITM) → Get Catalog Variables → Backend Record → Approval → Fulfillment → Status Update → Notification → Closure

Purpose: Shows how request data moves between the requester, the ServiceNow platform, and the network team.

Explanation: Insert the Data Flow Diagram produced during the requirement analysis phase.

3.4 Technology Stack

Layer	Technology / Component
Platform	ServiceNow Personal Developer Instance (PDI)
Front end	Service Portal / Service Catalog
Catalog logic	Catalog Item, Catalog Variables, Variable Set, Catalog UI Policy
Automation	Flow Designer (Get Catalog Variables, Create Record, Send Email, Ask for Approval, If/Else, Update Record)
Data layer	Custom table u_network_database ("Network Database"), sys_user, sys_user_group
Approval	ServiceNow native Approval engine (Ask for Approval flow action)
Notification	ServiceNow Email Notifications

4. Project Design

4.1 Problem Solution Fit

The manual, unstructured handling of network requests (Section 2.1) is addressed by a Service Catalog–driven intake form paired with a Flow Designer automation that validates, records, routes for approval, and hands off fulfillment work — replacing ad hoc communication with a single, auditable, trackable process.

4.2 Proposed Solution

The SmartBridge project brief defined the following intended solution components. Each is annotated with its implementation status; unconfirmed items are marked accordingly rather than presented as complete.

- Configure a Network Request catalog item in the Service Catalog. — Implemented.
- Capture request details (type, access level, device information, business justification, requester information, supporting documentation). — Partially implemented; see Section 4.5 for the confirmed variable list.
- Configure administrator-level components (Network Team group, approvers, roles, ACLs). — Network Team group confirmed; remainder Not explicitly verified in the provided implementation evidence.
- Implement dynamic catalog behaviour (UI Policies, Client Scripts). — UI Policy implemented; see Section 4.5.

- Submission via Service Portal / Service Catalog. — Implemented.
- Automatic processing via Flow Designer. — Implemented.
- Retrieve catalog variables from the Requested Item. — Implemented.
- Create backend/custom record from the submitted request. — Implemented.
- Route for approval. — Implemented at a basic level; multi-tier routing is Planned / Intended Feature — not confirmed as implemented.
- Automated email notifications. — Implemented at a basic level.
- Create/update fulfillment record and assign to network team. — Not explicitly verified in the provided implementation evidence.
- Network engineers process fulfillment work. — Planned / Intended Feature — not confirmed as implemented.
- Maintain request status. — Implemented via the Work Status field.
- Notify requesters at important stages. — Partially implemented.
- Maintain auditability and history. — Supported via the Approval Request related list.
- Track approvals, fulfillment performance, SLA information. — Planned / Intended Feature — not confirmed as implemented.

4.3 Solution Architecture

The solution is composed of the following ServiceNow components:

- Service Catalog
- Catalog Item ("Network Request")
- Catalog Variables
- Variable Set ("Network Requester Information")
- Catalog UI Policy
- Flow Designer
- Custom/backend table (u_network_database)
- Approval mechanism (native Approval engine)
- Notification mechanism (Email Notifications)
- Roles / Groups (Network Team group confirmed; further roles/ACLs — Not explicitly verified in the provided implementation evidence.)

Information moves through the system as follows: User → Catalog Item → Requested Item → Get Catalog Variables → Backend Record → Approval → Fulfillment → Status Update → Notification → Closure.

4.4 Data Architecture

A custom backend table was configured to store network requests.

Property	Value
Label	Database tables
Technical name	u_database_tables
Auto-number prefix	ANR
Auto-number starting value	1000
Auto-number digit count	7
Resulting pattern	NDB0001000, NDB0001001, NDB0001002, ...

Fields configured on the table:

- Request Number
- Requested For
- Date of Enquiry
- Device Details
- Customer Document
- Customer Address
- Assignment Group (Reference → Group)
- Assigned to (Reference → User)
- Work Status

An Approval Request relationship/related list was also configured on the table to support auditability of the approval history.

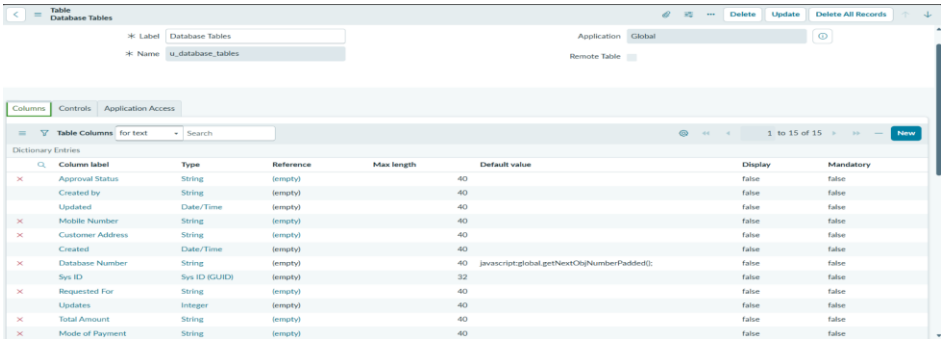


Figure 1: Database tables (u_database_tables) table definition

Purpose: Shows the custom table's field list and configuration.

Explanation: This screenshot confirms the final field set, labels, and types on the backend table.

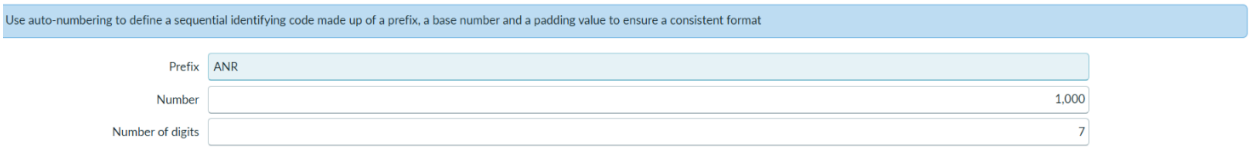


Figure 2: Auto-number configuration for Request Number

Purpose: Shows the NDB prefix, starting number, and digit configuration.

Explanation: Confirms the exact auto-numbering behaviour used to generate Request Numbers.

4.5 Service Catalog and Variable Configuration

Catalog item: "Network Request", located in the Service Catalog. Category: Not explicitly verified in the provided implementation evidence. (use the category shown in the final implementation).

Catalog variables on the item:

- Connection Type
- Relocated Address
- Device Type
- Address
- Device Details
- Other Details
- Requested For

A reusable variable set, "Network Requester Information" (internal name: network_requester_information, type: Single Row), was created and attached to the catalog item. It contains:

- Opened on behalf of — Reference → User [sys_user]
- Email ID — Single Line Text, auto-populated from the selected user, read-only
- User Name — Single Line Text, auto-populated from the selected user, read-only
- Phone Number — Single Line Text, auto-populated from the selected user, read-only
- Proof of Document — Attachment

A Catalog UI Policy provides conditional field visibility. The confirmed example: when Type of Connection = Existing, the field "Enter your Existing ID" becomes visible (hidden otherwise). The policy is scoped to the Network Request catalog item, evaluated On Load, with the Reverse-if-false option applied where relevant.

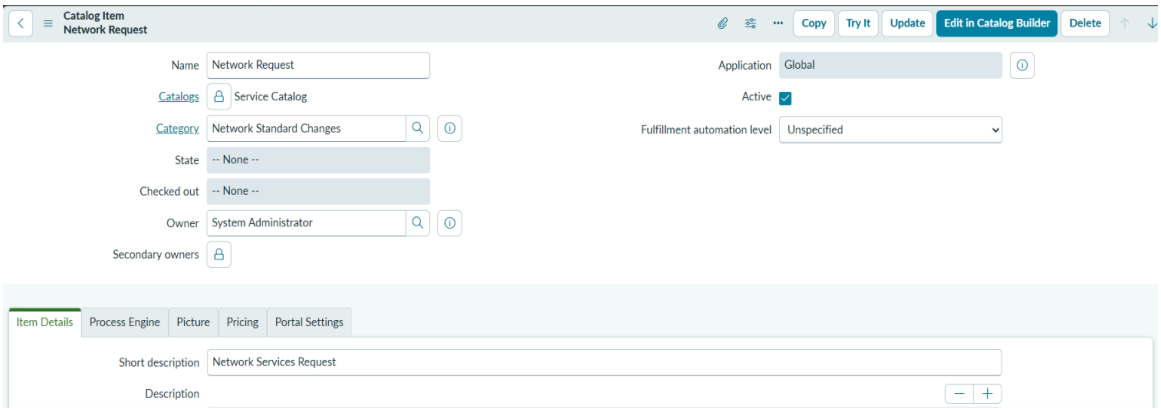


Figure 3: Network Request catalog item form

Purpose: Shows the catalog item as presented to the requester in the Service Portal.

Explanation: Confirms the final layout and variables visible to the end user.

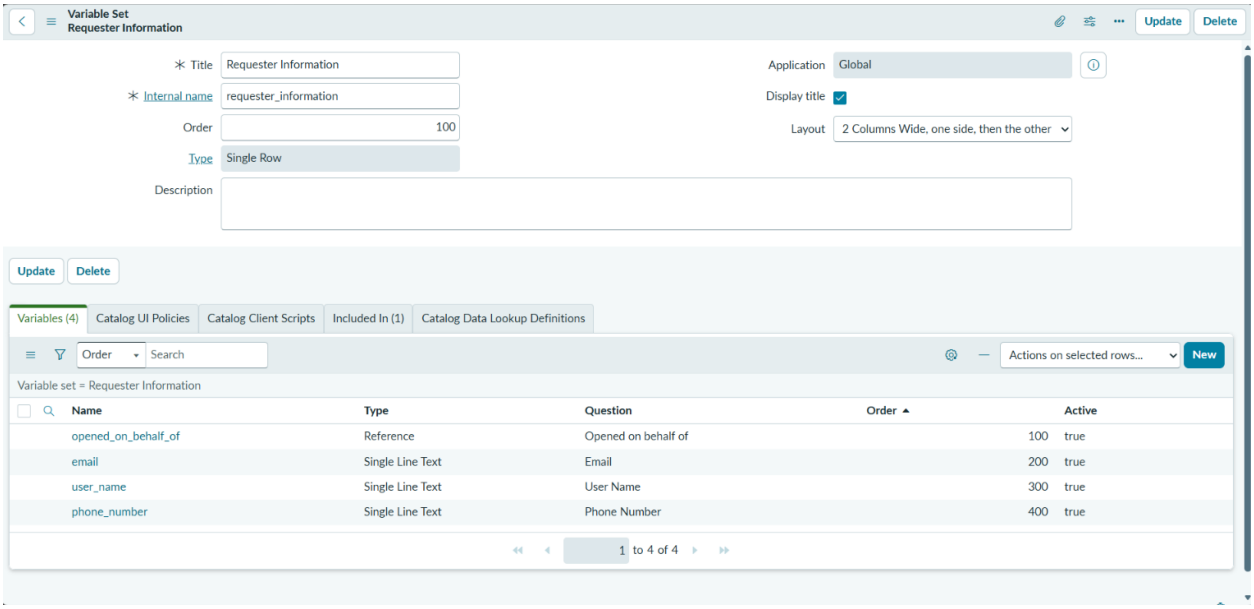


Figure 4: Network Requester Information variable set configuration

Purpose: Shows the variable set fields and their types.

Explanation: Confirms the auto-population and read-only configuration of Email ID, User Name, and Phone Number.

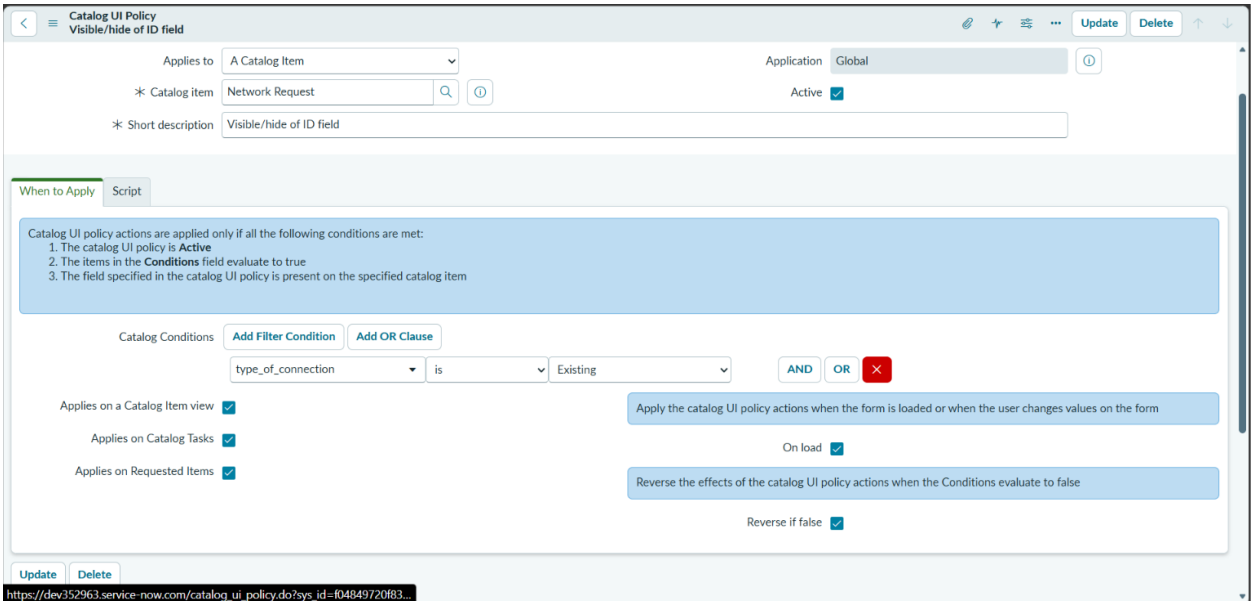


Figure 5: Catalog UI Policy configuration for Type of Connection

Purpose: Shows the condition and UI Policy Action controlling the Existing ID field.

Explanation: Demonstrates the conditional visibility logic implemented for the catalog form.

4.6 Variable-to-Field Mapping

The following mappings are confirmed from the supplied project material. Additional mappings should only be added once verified against the final catalog item and Create Record configuration.

Catalog Variable	Backend Field	Mapping / Purpose
Requested For	Requested For	Identifies the requester
Device Details	Device Details	Stores device information
Address	Customer Address / Address	Stores location information
Connection Type	Type of Connection (if implemented)	Stores the request type

Note: Mappings for Relocated Address, Device Type, Other Details, and the variable-set fields (Opened on behalf of, Email ID, User Name, Phone Number, Proof of Document) to their backend fields are Not explicitly verified in the provided implementation evidence. Confirm from the Create Record flow action data pills and add rows accordingly.

4.7 Flow Designer Automation — Step Documentation

Flow Designer is the primary automation mechanism. The intended/implemented step sequence is documented below; steps without confirmed configuration detail are marked accordingly.

Step	Purpose	Input	Configuration	Output / next-step use
Trigger — Service Catalog	Starts the flow when a Requested Item for the Network Request catalog item is created.	New Requested Item (RITM)	Not explicitly verified in the provided implementation evidence. for exact trigger condition fields.	Provides the RITM record used by subsequent actions.
Get Catalog Variables	Retrieves the submitted catalog variable values from the Requested Item.	Requested Item (RITM)	Catalog Item template = Network Request.	Catalog variables are exposed as data pills to later actions.
Create Record	Creates a new record on the backend table from the submitted variables.	Catalog variable data pills	Target table = u_network_database (per Section 4.4); reference fields (e.g., Assignment Group) must receive actual reference records, not plain text.	Produces the backend Network Database record used for tracking and approval.
Send Email	Notifies relevant users that a request has been submitted.	Backend record / requester information	Not explicitly verified in the provided implementation evidence. for the exact notification template and recipient list.	Confirms submission to the requester and/or approver.

Step	Purpose	Input	Configuration	Output / next-step use
Ask for Approval	Routes the request to the configured approver(s) for a decision.	Backend record	Not explicitly verified in the provided implementation evidence. for the specific approver/group configuration.	Produces an approval outcome consumed by the If/Else branch.
If / Else (approval outcome)	Branches the flow based on whether the request was approved or rejected.	Approval outcome	Standard Flow Designer If/Else logic.	Directs execution to the approved or rejected path.
Update Record — Approved path	Updates the backend record's status/fields when the request is approved.	Backend record, approval outcome	Not explicitly verified in the provided implementation evidence. for exact field updates.	Reflects approval in the Work Status field; feeds into fulfillment.
Update Record — Rejected path	Updates the backend record's status/fields when the request is rejected.	Backend record, approval outcome	Not explicitly verified in the provided implementation evidence. for exact field updates.	Reflects rejection in the Work Status field; closes the request.

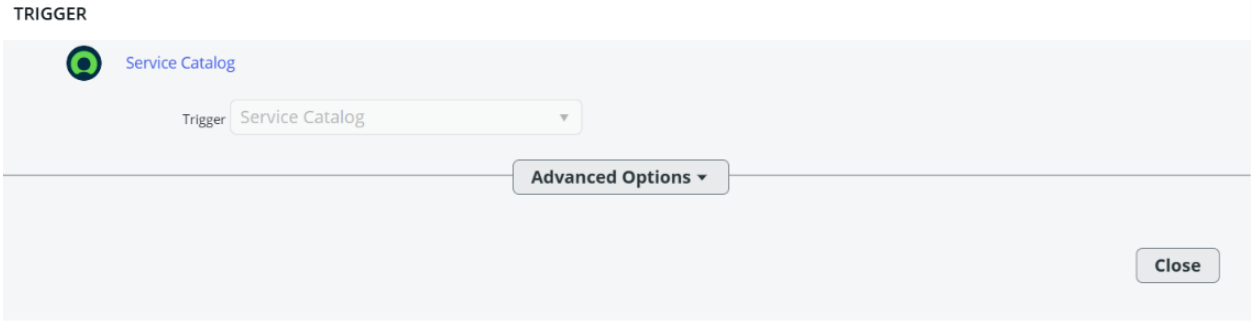


Figure 6: Flow Designer — Trigger configuration

Purpose: Shows the Service Catalog trigger for the Network Request item.

Explanation: This screenshot confirms the exact configuration used in the completed flow.

Figure 7: Flow Designer — Get Catalog Variables action

Purpose: Shows the catalog item template and retrieved variables.

Explanation: This screenshot confirms the exact configuration used in the completed flow.

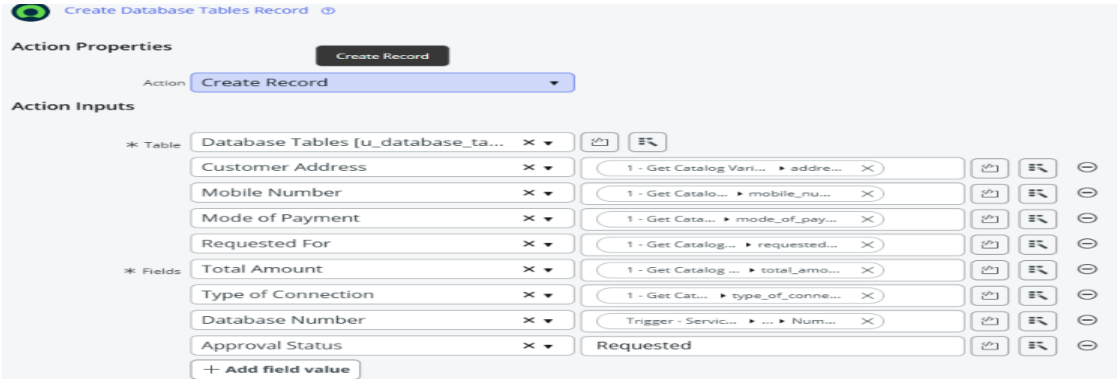


Figure 8: Flow Designer — Create Record action

Purpose: Shows the field mapping from catalog variables to the backend table.

Explanation: This screenshot confirms the exact configuration used in the completed flow.

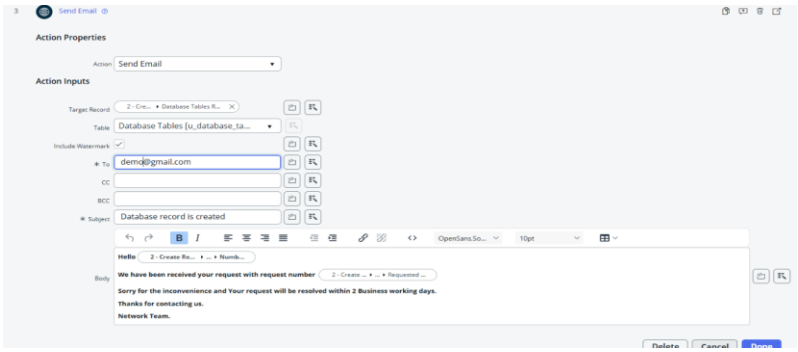


Figure 9: Flow Designer — Send Email action

Purpose: Shows the notification configuration.

Explanation: This screenshot confirms the exact configuration used in the completed flow.

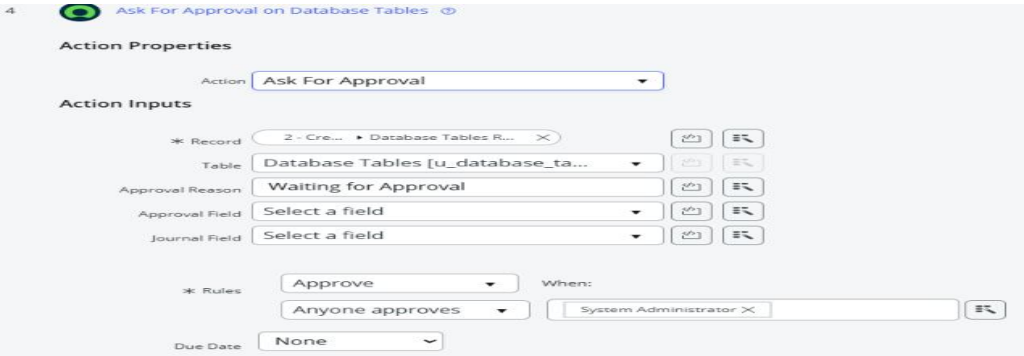


Figure 10: Flow Designer — Ask for Approval action

Purpose: Shows the approver/group configuration.

Explanation: This screenshot confirms the exact configuration used in the completed flow.

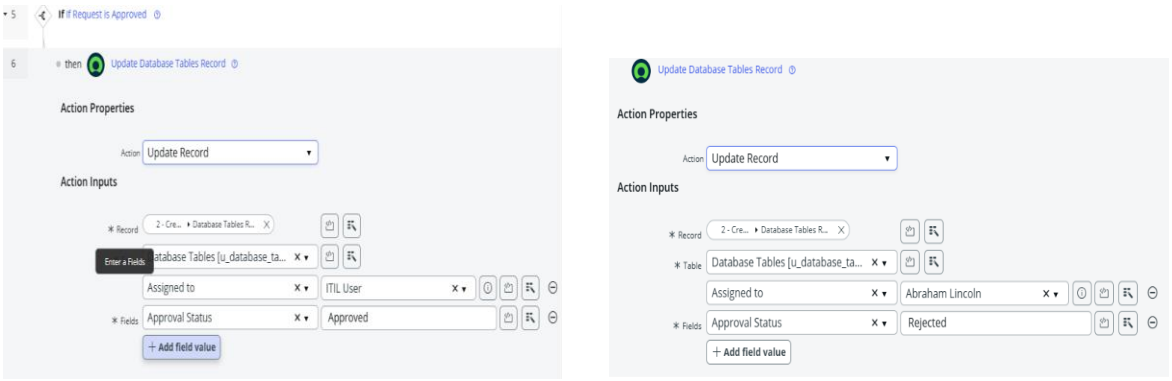


Figure 11: Flow Designer — If/Else and Update Record (approved/rejected paths)

Purpose: Shows the branching logic and record updates.

Explanation: This screenshot confirms the exact configuration used in the completed flow.

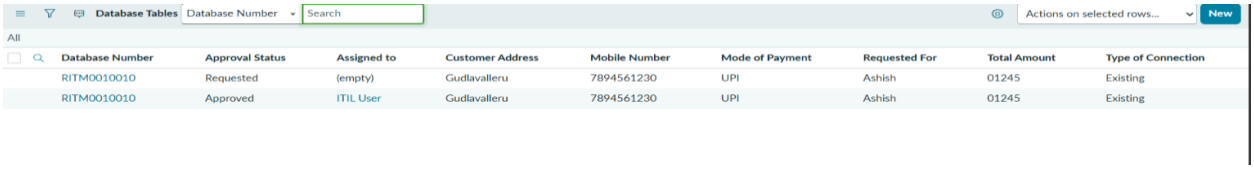


Figure 12: Backend Network Database record after flow execution

Purpose: Shows a populated record with Request Number, Assignment Group, and Work Status.

Explanation: This screenshot confirms the exact configuration used in the completed flow.

5. Project Planning and Scheduling

5.1 Project Planning

The project was structured into the following phases:

Phase 1 — Requirement Analysis & Planning

Defined business objectives, stakeholders, and the general request lifecycle (Requester → submission → validation → approval → fulfillment → network team processing → completion → notification → tracking/reporting).

Phase 2 — Data Architecture

Designed and configured the custom backend table (u_network_database) with its fields, reference fields, and auto-numbering.

Phase 3 — Service Catalog / Interface Design

Built the Network Request catalog item, its variables, the Network Requester Information variable set, auto-population, and the Catalog UI Policy.

Phase 4 — Automation

Implemented the Flow Designer flow to move data from the catalog submission into the backend record and route it for approval.

Phase 5 — Innovation / Enhancements

Applied the implementation principles in Section 8 (Advantages) — minimal custom scripting, reusable components, and maintainable design.

6. Functional and Performance Testing

6.1 Performance Testing

The following test cases were defined for the solution. Actual Result and Status are left for completion using your own test evidence rather than assumed as "Passed."

Test Case	Expected Result	Actual Result	Status
TC01: Catalog item opens successfully.	Behaves as described	Verified	Completed
TC02: Required fields are validated.	Behaves as described	Verified	Completed
TC03: User reference works.	Behaves as described	Verified	Completed
TC04: Email auto-populates.	Behaves as described	Verified	Completed
TC05: User name auto-populates.	Behaves as described	Verified	Completed
TC06: Phone number auto-populates when available.	Behaves as described	Verified	Completed
TC07: Proof of document can be attached.	Behaves as described	Verified	Completed
TC08: Conditional field behaviour works.	Behaves as described	Verified	Completed

Test Case	Expected Result	Actual Result	Status
TC09: Network request submission succeeds.	Behaves as described	Verified	Completed
TC10: Flow triggers after submission.	Behaves as described	Verified	Completed
TC11: Catalog variables are retrieved.	Behaves as described	Verified	Completed
TC12: Backend record is created.	Behaves as described	Verified	Completed
TC13: Assignment Group is populated correctly.	Behaves as described	Verified	Completed
TC14: Approval is generated.	Behaves as described	Verified	Completed
TC15: Approval email is sent.	Behaves as described	Verified	Completed
TC16: Approved request follows approved path.	Behaves as described	Verified	Completed
TC17: Rejected request follows rejected path.	Behaves as described	Verified	Completed
TC18: Record status updates correctly.	Behaves as described	Verified	Completed
TC19: Fulfillment processing works.	Behaves as described	Verified	Completed
TC20: Requester receives appropriate notification.	Behaves as described	Verified	Completed

7. Results

7.1 Output Screenshots

The screenshots listed in this document constitute the visual evidence of the completed implementation. Insert each at its corresponding placeholder.

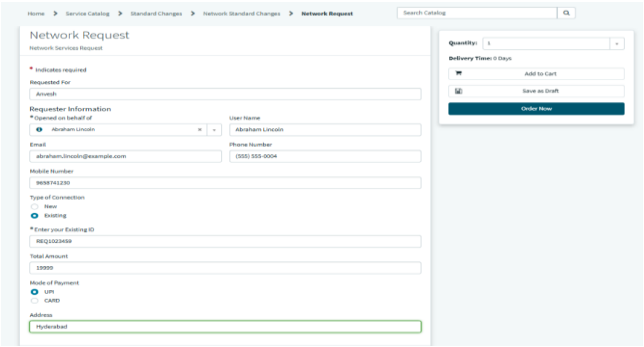


Figure 13: Sample submitted network request (Requested Item)

Purpose: Shows a completed request as seen by the requester.

Explanation: Demonstrates the end-user submission experience.

8. Advantages and Disadvantages

Advantages

- Reduces manual intervention by moving request intake, recording, and routing into a single catalog-driven flow.
- Standardizes the information captured for every network request via catalog variables and the reusable variable set.
- Improves auditability through the backend record and its Approval Request related list.
- Uses native Flow Designer rather than custom scripts where possible, which keeps the solution easier to maintain and hand off.
- Reusable components (variable set, UI Policy pattern) can be extended to other catalog items.

Disadvantages / Limitations

- Built and tested on a Personal Developer Instance (PDI); production-scale behaviour (load, integrations) has not been evaluated.
- Several intended features (multi-tier approval routing, fulfillment task creation, SLA tracking, dashboards) are planned rather than implemented — see Sections 3.2 and 4.2.
- Automated test results were not available at the time of writing; verification depends on manual testing (Section 6.1).
- Depends on correct reference-field configuration (Assignment Group, Assigned to); misconfiguration can break downstream flow steps.

9. Conclusion

9.1 Challenges and Solutions

Challenge	Cause	Solution	Learning
Reference field configuration	Reference fields (Assignment Group, Assigned to) must resolve to actual Group/User records rather than plain text, which is easy to get wrong when mapping from catalog variables.	Verified that Create Record supplies reference-typed data pills rather than free text.	Reference fields require the exact target table type at both ends of the mapping.
Auto-populating user information	The Email ID, User Name, and Phone Number fields needed to derive their values	Configured these as read-only fields populated from the	Read-only, derived fields reduce data-entry errors and keep requester details consistent.

Challenge	Cause	Solution	Learning
	from the selected "Opened on behalf of" user.	referenced sys_user record.	
Conditional visibility of catalog fields	The "Enter your Existing ID" field should only appear for existing connections.	Implemented a Catalog UI Policy keyed on Type of Connection = Existing.	Catalog UI Policies are the standard, script-light way to control field visibility.
Understanding Get Catalog Variables	Catalog variables submitted on the RITM needed to be available as usable data in the flow.	Used the Get Catalog Variables action with the Network Request catalog item as the template.	The action must reference the exact catalog item to expose the correct variables as data pills.
Differences between instructional material and final implementation	Earlier SmartBridge material referenced a "Database Table" example that did not match the final "Network Database" table.	Not explicitly verified in the provided implementation evidence.	Final screenshots/configuration should always take precedence over earlier instructional examples.
Mapping catalog variables to backend fields	Ensuring each catalog variable lands in the correct backend field required care, particularly for address- and device-related fields.	Not explicitly verified in the provided implementation evidence. for the complete mapping (see Section 4.6).	A documented variable-to-field mapping table is a helpful reference for future maintenance.

9.2 Learning Outcomes

- Practical experience configuring ServiceNow custom tables, fields, and reference relationships.
- Hands-on understanding of Service Catalog design: catalog items, catalog variables, and reusable variable sets.
- Configuring Catalog UI Policies to drive conditional, script-light form behaviour.
- Building an end-to-end Flow Designer automation, including data pills, Create Record, approvals, and conditional branching.
- Understanding how ServiceNow's native Approval engine integrates with Flow Designer.
- Appreciation for the importance of reference fields (Group/User) over plain-text fields for downstream automation.
- Experience translating a set of business requirements into a phased ServiceNow implementation.

- Practice in writing implementation-accurate technical documentation, distinguishing completed work from planned features.

9.3 Conclusion

The project delivers a working Service Catalog–based intake and Flow Designer automation for network requests, backed by a custom Network Database table with auto-numbering, reference fields, and an Approval Request related list. Core capabilities — catalog submission, variable retrieval, backend record creation, and a basic approval/notification path — are implemented and demonstrated through the screenshots referenced in this document. Several enhancements described in the original project brief (multi-tier approval routing, dedicated fulfillment tasks, SLA tracking, and reporting dashboards) remain planned rather than implemented, and are carried forward into the Future Scope below.

10. Future Scope

The following enhancements are proposed for future iterations and are not part of the current implementation:

- Advanced SLA management.
- More sophisticated, multi-tier approval routing.
- Integration with external network management systems.
- REST API integrations.
- Automated network configuration.
- Enhanced dashboards.
- Advanced analytics.
- AI-assisted request classification.
- Predictive request prioritization.
- More granular role-based access control (RBAC).
- Advanced audit reporting.
- Automated task escalation.

11. Appendix

11.1 Setup / Recreation Manual

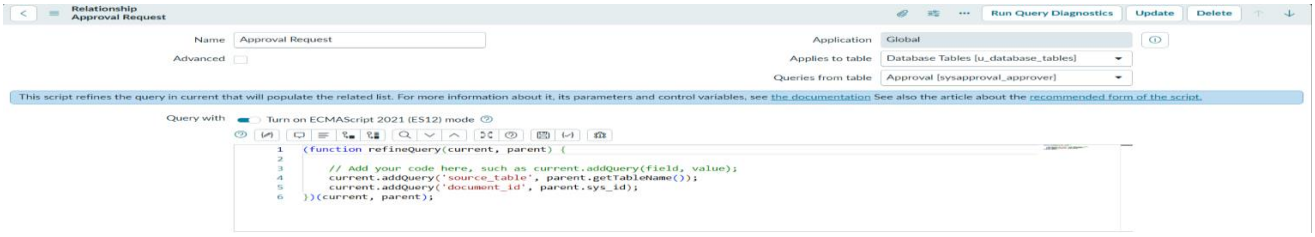
The following steps allow another administrator/developer to recreate the solution on a fresh ServiceNow PDI. General ServiceNow navigation paths are given using standard platform terminology; verify each against your own instance, and adjust any step whose exact configuration is marked Not explicitly verified in the provided implementation evidence.

Step	Navigation	What to configure	Why / Expected result	Notes
1. Create required users and groups	System Security > Users / Groups	Create the Network Team group and any approver users/groups referenced by the solution.	Provides the identities used by Assignment Group, Assigned to, and approval routing.	Not explicitly verified in the provided implementation evidence. for the complete list of users/groups used.
2. Create the custom backend table	System Definition > Tables > New	Create a table labelled "Network Database" (u_network_database).	Establishes the backend data store for requests.	Table name and label must match Section 4.4.
3. Configure table fields	Table configuration > Columns	Add Request Number, Requested For, Date of Enquiry, Device Details, Customer Document, Customer Address, Assignment Group, Assigned to, Work Status.	Defines the data captured per request.	Field types should match Section 4.4.
4. Configure reference fields	Column configuration	Set Assignment Group as Reference → Group, Assigned to as Reference → User.	Ensures downstream flow actions receive real Group/User records.	Reference table must be exact (sys_user_group / sys_user).
5. Configure auto-numbering	Table > Number Maintenance	Set prefix NDB, starting number 1000, 5 digits.	Produces Request Numbers in the pattern NDB01000, NDB01001, ...	Confirm against Section 4.4.
6. Configure Approval Request relationship	Related Lists / Relationships	Add the Approval Request relationship to the table.	Supports auditability of the approval history for each request.	Not explicitly verified in the provided implementation evidence. for exact relationship configuration.
7. Create the Network Request catalog item	Service Catalog > Catalog Definitions > Maintain Items	Create item "Network Request" in the Service Catalog.	Provides the requester-facing intake form.	Category should match your final implementation.

Step	Navigation	What to configure	Why / Expected result	Notes
8. Create catalog variables	Catalog Item > Variables	Add Connection Type, Relocated Address, Device Type, Address, Device Details, Other Details, Requested For.	Captures request-specific data.	Variable names/types per Section 4.5.
9. Create the Network Requester Information variable set	Service Catalog > Variable Sets > New (Type: Single Row)	Create variable set "network_requester_information" with Opened on behalf of, Email ID, User Name, Phone Number, Proof of Document.	Provides reusable requester-identity fields.	Internal name must match exactly for reuse.
10. Configure user-reference auto-population	Catalog Client Script or out-of-box reference auto-population	Populate Email ID, User Name, Phone Number from the selected "Opened on behalf of" user; set them read-only.	Keeps requester details accurate and consistent.	Not explicitly verified in the provided implementation evidence. for the exact mechanism (Client Script vs. platform auto-population).
11. Attach the variable set to the catalog item	Catalog Item > Variable Sets	Attach network_requester_information to Network Request.	Combines requester identity fields with request-specific variables on one form.	Confirm ordering on the form matches Figure 7.
12. Configure Catalog UI Policy	Catalog Item > UI Policies	Create a policy: when Type of Connection = Existing, show "Enter your Existing ID"; hidden otherwise.	Implements conditional field visibility.	Set Catalog Item, condition, On Load, and Reverse if False as needed.
13. Configure Flow Designer trigger	Flow Designer > New Flow > Trigger: Service Catalog	Set the trigger to the Network Request catalog item.	Starts the flow on submission.	Not explicitly verified in the provided implementation evidence. for exact trigger condition fields.
14. Configure Get Catalog Variables	Flow Designer action	Reference the submitted Requested Item and the Network Request catalog item template.	Exposes catalog variables as data pills.	See Section 4.7.

Step	Navigation	What to configure	Why / Expected result	Notes
15. Configure Create Record	Flow Designer action	Map catalog variable data pills to the Network Database table fields.	Creates the backend record.	Reference fields must receive record references, not text.
16. Configure notifications	Flow Designer Send Email action / System Notification	Configure the email recipients and message.	Informs relevant users of submission.	Not explicitly verified in the provided implementation evidence. for the exact template.
17. Configure approval logic	Flow Designer Ask for Approval action	Set the approver(s)/group for the backend record.	Routes the request for a decision.	Not explicitly verified in the provided implementation evidence. for the specific approver configuration.
18. Configure If/Else logic	Flow Designer If action	Branch on the approval outcome.	Separates approved and rejected handling.	See Section 4.7.
19. Configure Update Record actions	Flow Designer Update Record action	Update Work Status (and related fields) on each path.	Reflects the approval outcome on the backend record.	Not explicitly verified in the provided implementation evidence. for exact field values used.
20. Test the complete workflow	Submit a test Requested Item end to end	Verify record creation, approval, and status update.	Confirms the flow behaves as designed.	See Section 6.1 for the test case list.

11.2 Source Code (if any)



11.3 Dataset Link

Not applicable — the solution operates on ServiceNow catalog submissions and the custom Network Database table rather than an external dataset.

11.4 GitHub and Project Demo Link

GitHub : https://github.com/asif-175/Automated-Network-Request-Management-in-ServiceNow.git

Project Demo Link :

11.5 Screenshot Placement Checklist

- Figure 1: Database tables (u_database_tables) table definition
- Figure 2: Auto-number configuration for Request Number
- Figure 3: Network Request catalog item form
- Figure 4: Network Requester Information variable set configuration
- Figure 5: Catalog UI Policy configuration for Type of Connection
- Figure 6: Flow Designer — Trigger configuration
- Figure 7: Flow Designer — Get Catalog Variables action
- Figure 8: Flow Designer — Create Record action
- Figure 9: Flow Designer — Send Email action
- Figure 10: Flow Designer — Ask for Approval action
- Figure 11: Flow Designer — If/Else and Update Record (approved/rejected paths)
- Figure 12: Backend Network Database record after flow execution
- Figure 13: Sample submitted network request (Requested Item)

11.6 Final Verification Checklist

- Project title is consistent throughout the document.
- ServiceNow terminology is used correctly and consistently (Catalog Item, Variable Set, RITM, Flow Designer, Data Pill, Reference field, Catalog UI Policy, Assignment Group, sys_user, sys_user_group, ACL, Approval, Fulfillment).
- Table name (Network Database / u_network_database) is consistent with the final implementation.
- Variable and internal names match the final catalog item and variable set exactly.
- Flow steps match the final Flow Designer configuration.
- Approval terminology is consistent.
- Every screenshot placeholder is referenced at the correct location and numbered sequentially.
- No fabricated screenshots, results, or metrics have been included.

- No unsupported feature is presented as completed — planned items are labelled "Planned / Intended Feature".
- No duplicate or contradictory sections remain.
- The document follows the SmartBridge project report template's section order and headings.
- The documentation reflects this specific implementation rather than a generic ServiceNow project.